

Optical Flicker Generator — Interface & Protocol Reference

Overview

The device exposes three interfaces:

Interface	Transport	Port	Format
ASCII shell	USB Serial (CDC)	—	Line-oriented text
ASCII shell	Ethernet / Telnet	TCP 23	Same as Serial
HTTP API	Ethernet	TCP 80	JSON

Telnet access can be enabled/disabled in configuration and is disabled by default. Parameter semantics are consistent across Serial, Telnet, and HTTP interfaces. The output is two hardware channels driven in lockstep with the same PWM parameters; there is no per-channel control.

ASCII Shell (Serial + Telnet)

Framing

- One command per line, terminated by `\n` or `\r\n`.
- Command verb is **case-insensitive**; arguments are case-sensitive where noted.
- Response is a single line terminated by `\r\n`.
- Maximum accepted line length is 127 characters; additional characters are ignored until newline, then the truncated line is executed.
- Partial lines with no new data for 5 s are silently discarded.

Response format

```
OK                # success, no payload
OK <payload>      # success with data
ERROR <reason>    # failure; reason is human-readable
```

Errors always begin with the literal string `ERROR` followed by a short description. Scripts should match on the prefix.

Commands

identify

Query device identity.

```
→ identify
```

```
← OK FLICKER_DEVICE 1.0 A0B1C2D3E4F56789A0B1C2D3E4F56789
```

Payload: <device_type> <firmware_version> <id>

<id> is the device's 128-bit hardware serial number encoded as 32 uppercase hex chars.

mode [value]

Query or set the operating mode.

```
→ mode
← OK flicker

→ mode flicker
← OK
```

Valid modes: off, constant, flicker, sinus

Changing mode keeps existing frequency/duty/intensity values; only the mode changes.

Mode	Description
off	LED output disabled
constant	Continuous illumination at the configured intensity
flicker	Square-wave envelope at configured frequency and duty cycle
sinus	Sinusoidal envelope at configured frequency

frequency / freq [value]

Query or set the flicker/sinus frequency in Hz.

```
→ frequency
← OK 50

→ freq 10
← OK

→ freq 0
← ERROR out of range (1-500 Hz)
```

Range: 1–500 Hz

Setting frequency does not change mode.

dutycycle / duty [value]

Query or set the duty cycle percentage (flicker mode only; stored for other modes too).

```
→ duty
← OK 50
```

```
→ duty 30
← OK

→ duty 5
← ERROR out of range (10-90 %)
```

Range: 10–90 %

intensity/int [value]

Query or set the LED intensity percentage.

```
→ intensity
← OK 100

→ int 75
← OK
```

Range: 0–100 %

carrier [value]

Query or set the PWM carrier frequency in Hz. The carrier drives intensity modulation in constant and sinus modes; it has no effect in flicker mode. The value is saved immediately.

```
→ carrier
← OK 20000

→ carrier 10000
← OK

→ carrier 500
← ERROR out of range (1000-50000 Hz)
```

Range: 1 000–50 000 Hz

screensaver [value]

Query or set the display screensaver timeout in seconds. The value is saved immediately.

```
→ screensaver
← OK 10

→ screensaver 60
← OK

→ screensaver 0
← ERROR out of range (1-600 s)
```

Range: 1–600 s

Error reference

Message	Cause
ERROR empty command	Line contained no token
ERROR unknown command	Verb not recognised
ERROR invalid argument	Argument not parseable
ERROR unknown mode (off constant flicker sinus)	Unrecognised mode name
ERROR out of range (1-500 Hz)	Frequency outside 1–500 Hz
ERROR out of range (10-90 %)	Duty cycle outside 10–90 %
ERROR out of range (0-100 %)	Intensity outside 0–100 %
ERROR out of range (1000-50000 Hz)	Carrier frequency outside 1 000–50 000 Hz
ERROR out of range (1-600 s)	Screensaver timeout outside 1–600 s

HTTP / JSON API

Base URL: `http://<device-ip>/`

Responses use `Content-Type: application/json` and `Connection: close`.

GET /api/identify

```
{
  "device": "FLICKER_DEVICE",
  "firmware": "1.0",
  "id": "A0B1C2D3E4F56789A0B1C2D3E4F56789"
}
```

GET /api/state

Returns the current mode and all parameters.

```
{
  "mode": "flicker",
  "frequency": 50,
  "duty cycle": 50,
  "intensity": 100
}
```

POST /api/off

Turns the LED off. No request body required.

```
{ "ok": true }
```

POST /api/constant

Sets constant mode. Optional JSON body applies parameters before switching.

```
{ "intensity": 75 }  
→  
{ "ok": true }
```

POST /api/flicker

Sets flicker mode. Optional JSON body applies parameters before switching.

```
{ "frequency": 10, "dutycycle": 30, "intensity": 80 }  
→  
{ "ok": true }
```

JSON keys: `frequency` (Hz, 1–500), `dutycycle` (% , 10–90), `intensity` (% , 0–100). All are optional; omit any key to keep the current value.

POST /api/sinus

Sets sinus mode. Optional JSON body.

```
{ "frequency": 25, "intensity": 100 }  
→  
{ "ok": true }
```

GET /api/frequency

```
{ "frequency": 50 }
```

POST /api/frequency

```
{ "value": 25 }  
→  
{ "ok": true }
```

GET /api/dutycycle / POST /api/dutycycle

Same pattern. POST body: { "value": 40 }.

GET /api/intensity / POST /api/intensity

Same pattern. POST body: { "value": 80 }.

HTTP error responses

```
{ "error": "Not found" }           // 404 – unknown path  
{ "error": "Bad request" }        // 400 – unsupported /api request (unknown  
endpoint or wrong method)
```

Note: the mode-set POST endpoints (`/api/flicker`, `/api/sinus`, etc.) do **not** return range-validation errors for individual parameters — out-of-range values are silently clamped. Use the ASCII shell if you need explicit range errors.

Web UI

GET `/` — HTML control page for interactive use.

GET `/config` — HTML network configuration page (DHCP, Telnet enable/disable, static IP/gateway/subnet, PWM carrier frequency).

POST `/config` — Submit configuration form. Changes take effect on next boot for network settings.

Network configuration

Configured via the `/config` web page. Stored fields:

Field	Default
DHCP	enabled
Telnet (TCP 23)	disabled
Static IP	0.0.0.0
Gateway	0.0.0.0
Subnet	0.0.0.0
PWM carrier Hz	20000

The PWM carrier frequency (1 000–50 000 Hz) controls the intensity modulation rate in `constant` and `sinus` modes. It has no effect in `flicker` mode. When DHCP is disabled, the configured static IP, gateway, and subnet values are used at boot.

Network changes take effect after a power cycle.

mDNS discovery

The device advertises itself on the LAN via **mDNS/DNS-SD** (UDP port 5353).

Item	Value
Hostname	<code>flicker - + last 6 hex digits of device serial</code> (e.g. <code>flicker-a1b2c3.local</code>)
Service	<code>_http._tcp</code> on port 80

After the device has an IP address, open `http://flicker-<suffix>.local/` in a browser, or ping the hostname:

```
ping flicker-a1b2c3.local
```

Linux: `avahi-browse -a | grep -i flicker`

macOS: mDNS is built in; Safari Bonjour bookmarks also list `_http._tcp` services.

Windows: Built-in mDNS support varies by version. Install [Bonjour Print Services](#) if `.local` names do not resolve.

mDNS is always enabled when Ethernet has a valid IP. No configuration required.

Set button

Short press on set button wakes the display from screensaver.

Hold set button for **10 s** to perform a factory reset: network settings and PWM carrier Hz are cleared to defaults.